Assignment 7:

Name: Leon Lobo

Admission No: U24AI043

The 8 queen's problem is stated as: Place 8 queens on a chessboard such that no two queens attack each other. Considering this problem, complete the following tasks considering a valid heuristic function with justification.

Q1) Generate 50 random initial boards. Run steepest -ascent hill climbing on each and record initial heuristic value, final heuristic value, number of steps taken and status solved or fail. Prove the presence of a local minimum.

Code:

```python
import random
iterations=50

def generate_board():
    #queens=[3,7,0,4,6,1,5,2]
    queens=[random.randint(0,7) for i in range(8)]
    return queens

def h(queens):
    attacks=0
    for i in range(8):
        for j in range(i+1,8):
            if queens[i]==queens[j] or (i-queens[i])==(j-queens[j]) or (i+queens[i])==(j+queens[j]):
                #print(f"({queens[i]},{i}) ({queens[j]},{j})")
                attacks+=1
    return attacks


def steepest_ascent(queens):
    state=queens.copy()
    neighbour=state.copy()
    steps=0
    while True:
        attacks=100
        for i in range(8):
            state=queens.copy()
            for j in range(8):
                state[i]=j
                #print(state)
                #print(h(state))
                if(h(state)<attacks):
                    attacks=h(state)
                    neighbour=state.copy()
                    if attacks==0:
                        return neighbour, attacks, steps
                    #print(neighbour)
```

```python
def steepest_ascent(queens):
                if(h(state)<attacks):
                    attacks=h(state)
                    neighbour=state.copy()
                    if attacks==0:
                        return neighbour, attacks, steps
                    #print(neighbour)

        steps+=1
        if h(queens)<=attacks:
            return queens, h(queens), steps
        #print(h(queens),attacks,queens,neighbour)
        queens=neighbour.copy()



def main():
    success=0
    avg_steps=0
    for i in range(iterations):
        print(f"Board {i+1}:")
        queens=generate_board()
        ans,attacks,cost=steepest_ascent(queens)
        print(queens,h(queens))
        print(ans,attacks,cost)
        avg_steps+=cost
        if attacks==0:
            success+=1
            print("Solved\n")
        else:
            print("Failed\n")
    avg_steps/=iterations
    print("===== Steepest Ascent =====")
    print(f"Success Rate={success/iterations*100}%    Average Steps Taken={avg_steps}\n")

if __name__=="__main__":
    main()
```

Output:

```
Board 1:
[2, 6, 3, 3, 3, 1, 5, 6] 9
[2, 6, 3, 7, 4, 1, 5, 0] 1 5
Failed

Board 2:
[3, 1, 1, 4, 2, 0, 1, 6] 7
[3, 1, 7, 4, 2, 0, 2, 6] 1 3
Failed

Board 3:
[7, 6, 6, 3, 6, 0, 7, 5] 5
[7, 0, 6, 3, 6, 0, 7, 5] 3 2
Failed

Board 4:
[6, 4, 2, 6, 2, 7, 2, 2] 10
[6, 4, 0, 0, 3, 7, 2, 2] 2 4
Failed

Board 5:
[1, 7, 1, 7, 5, 1, 2, 1] 12
[0, 3, 1, 7, 5, 6, 2, 0] 2 5
Failed

Board 6:
[0, 2, 6, 6, 1, 1, 3, 7] 5
[5, 2, 6, 6, 3, 1, 4, 7] 1 4
Failed

Board 7:
[1, 4, 0, 4, 0, 3, 1, 3] 8
[6, 4, 2, 0, 5, 7, 1, 3] 0 5
Solved

Board 8:
[6, 1, 0, 0, 0, 1, 4, 6] 9
[6, 1, 3, 0, 0, 7, 4, 2] 1 4
Failed

Board 9:
[6, 6, 4, 3, 3, 6, 2, 5] 8
[0, 1, 4, 7, 3, 6, 2, 5] 1 4
```

```
Failed

Board 10:
[5, 1, 2, 0, 4, 5, 7, 1] 9
[6, 4, 2, 0, 3, 5, 7, 1] 1 4
Failed

Board 11:
[4, 4, 6, 4, 3, 6, 4, 7] 10
[2, 4, 6, 0, 3, 1, 4, 7] 1 4
Failed

Board 12:
[5, 7, 2, 1, 7, 0, 3, 6] 3
[5, 2, 4, 1, 7, 0, 3, 6] 1 3
Failed

Board 13:
[7, 1, 0, 2, 7, 1, 5, 4] 7
[7, 2, 0, 3, 6, 1, 5, 4] 2 4
Failed

Board 14:
[1, 3, 2, 3, 6, 6, 1, 6] 9
[1, 4, 2, 0, 6, 3, 1, 6] 2 4
Failed

Board 15:
[2, 5, 2, 1, 6, 3, 6, 3] 8
[2, 5, 7, 4, 1, 3, 6, 3] 1 4
Failed

Board 16:
[5, 4, 1, 2, 2, 7, 1, 0] 7
[5, 4, 6, 3, 2, 7, 1, 0] 3 3
Failed

Board 17:
[2, 0, 5, 2, 1, 1, 7, 3] 5
[2, 0, 5, 6, 4, 1, 7, 3] 2 3
Failed

Board 18:
[3, 7, 6, 7, 1, 0, 6, 1] 8
[3, 0, 4, 7, 5, 2, 6, 1] 0 3
```

```
Solved

Board 19:
[7, 7, 1, 4, 0, 3, 7, 6] 8
[2, 5, 1, 4, 0, 3, 3, 7] 1 5
Failed

Board 20:
[7, 4, 3, 7, 5, 6, 5, 7] 9
[2, 4, 0, 7, 5, 1, 5, 7] 3 4
Failed

Board 21:
[7, 1, 3, 1, 4, 6, 5, 0] 5
[2, 5, 7, 1, 4, 6, 3, 0] 1 5
Failed

Board 22:
[2, 1, 6, 2, 1, 0, 3, 6] 9
[2, 1, 6, 4, 7, 0, 3, 5] 1 4
Failed

Board 23:
[1, 5, 7, 0, 4, 1, 2, 2] 6
[1, 5, 7, 0, 3, 6, 4, 2] 2 4
Failed

Board 24:
[5, 4, 4, 4, 5, 4, 1, 2] 14
[0, 4, 4, 7, 5, 3, 1, 6] 1 6
Failed

Board 25:
[5, 6, 0, 7, 2, 2, 1, 7] 7
[5, 3, 0, 7, 2, 4, 1, 7] 2 3
Failed

Board 26:
[1, 6, 1, 4, 3, 2, 5, 1] 13
[1, 4, 7, 3, 0, 2, 5, 1] 1 6
Failed

Board 27:
[1, 4, 1, 1, 3, 4, 1, 6] 14
[1, 4, 7, 5, 0, 2, 0, 6] 1 7
```

```
Failed

Board 28:
[2, 2, 3, 2, 4, 5, 6, 4] 9
[2, 0, 3, 1, 7, 5, 6, 4] 2 4
Failed

Board 29:
[1, 4, 6, 4, 1, 4, 0, 0] 8
[1, 3, 6, 7, 1, 4, 0, 5] 2 4
Failed

Board 30:
[7, 3, 4, 5, 5, 5, 4, 6] 8
[7, 3, 4, 0, 5, 5, 2, 6] 2 3
Failed

Board 31:
[7, 0, 4, 4, 6, 5, 4, 4] 11
[7, 0, 3, 1, 6, 5, 2, 4] 1 4
Failed

Board 32:
[1, 7, 1, 6, 1, 0, 0, 7] 6
[2, 5, 1, 6, 3, 0, 4, 7] 1 5
Failed

Board 33:
[0, 5, 0, 5, 6, 0, 3, 3] 7
[1, 6, 2, 5, 7, 0, 3, 3] 1 5
Failed

Board 34:
[4, 3, 4, 2, 4, 6, 2, 3] 9
[7, 3, 0, 2, 4, 6, 0, 3] 2 4
Failed

Board 35:
[5, 7, 3, 6, 3, 5, 4, 1] 5
[2, 7, 3, 6, 0, 5, 4, 1] 2 3
Failed

Board 36:
[1, 6, 4, 1, 1, 5, 6, 6] 7
[3, 6, 4, 1, 1, 5, 2, 6] 2 3
```

```
Board 37:
[5, 2, 3, 7, 3, 5, 4, 3] 12
[6, 2, 0, 7, 3, 5, 3, 1] 2 5
Failed

Board 38:
[6, 6, 5, 7, 2, 7, 7, 0] 8
[6, 1, 5, 7, 2, 4, 7, 0] 3 3
Failed

Board 39:
[0, 1, 5, 3, 5, 5, 5, 2] 13
[4, 1, 5, 3, 0, 7, 5, 2] 3 4
Failed

Board 40:
[4, 7, 7, 0, 5, 6, 0, 3] 4
[4, 1, 7, 2, 5, 6, 0, 3] 2 3
Failed

Board 41:
[5, 5, 0, 7, 7, 0, 5, 3] 9
[1, 6, 0, 7, 4, 0, 5, 3] 2 4
Failed

Board 42:
[3, 7, 6, 4, 3, 1, 6, 7] 6
[5, 7, 0, 4, 3, 1, 6, 2] 1 4
Failed

Board 43:
[5, 6, 0, 4, 0, 1, 4, 1] 7
[5, 3, 0, 4, 7, 1, 6, 2] 0 3
Solved

Board 44:
[6, 6, 5, 4, 7, 5, 6, 2] 9
[6, 3, 0, 4, 7, 5, 2, 2] 1 5
Failed

Board 45:
[1, 2, 2, 4, 3, 4, 3, 1] 10
[5, 2, 0, 7, 3, 4, 6, 1] 1 5
Failed
```

```
Board 46:
[0, 2, 2, 3, 5, 5, 6, 2] 16
[3, 7, 0, 4, 1, 5, 6, 2] 1 7
Failed

Board 47:
[4, 1, 5, 1, 6, 3, 4, 4] 9
[4, 1, 5, 2, 6, 3, 0, 4] 1 3
Failed

Board 48:
[6, 2, 6, 1, 3, 6, 0, 1] 8
[5, 2, 6, 1, 3, 4, 0, 7] 1 4
Failed

Board 49:
[4, 4, 1, 4, 4, 4, 6, 7] 14
[4, 6, 1, 5, 2, 0, 3, 7] 0 4
Solved

Board 50:
[0, 4, 7, 2, 3, 6, 1, 6] 6
[0, 4, 7, 5, 2, 6, 1, 3] 0 2
Solved

===== Steepest Ascent =====
Success Rate=10.0%    Average Steps Taken=3.74
```

Q2 Repeat the above experiment on first choice, random restart and simulated annealing variants of hill climbing and compare your results.

Code:

```python
import random
e=2.718
iterations=5

def generate_board():
    #queens=[3,7,0,4,6,1,5,2]
    queens=[random.randint(0,7) for i in range(8)]
    return queens

def h(queens):
    attacks=0
    for i in range(8):
        for j in range(i+1,8):
            if queens[i]==queens[j] or (i-queens[i])==(j-queens[j]) or (i+queens[i])==(j+queens[j]):
                #print(f"({queens[i]},{i}) ({queens[j]},{j})")
                attacks+=1
    return attacks


def first_choice(queens):
    state=queens.copy()
    steps=0
    while True:
        attacks=h(queens)
        steps+=1
        flag=0
        for i in range(8):
            state=queens.copy()
            for j in range(15):
                state[i]=random.randint(0,7)
                #print(state)
                #print(h(state))
                if(h(state)<attacks):
                    queens=state.copy()
                    #print(h(state))
                    if h(state)==0:
                        return queens, 0, steps
                    flag=1
                    break
                    #print(neighbour)

            if flag:
                break
        if h(queens)==attacks:
            return queens, h(queens), steps
```

```python
def random_restart(queens):
    state=queens.copy()
    neighbour=state.copy()
    steps=0
    while True:
        attacks=100
        for i in range(8):
            state=queens.copy()
            for j in range(8):
                state[i]=j
                #print(state)
                #print(h(state))
                if(h(state)<attacks):
                    attacks=h(state)
                    neighbour=state.copy()
                    if attacks==0:
                        return neighbour, attacks, steps
                    #print(neighbour)

        steps+=1
        if h(queens)<=attacks:
            queens=generate_board()
        #print(h(queens),attacks,queens,neighbour)
        else:
            queens=neighbour.copy()


def schedule(t):
    T0=100
    a=0.9
    return T0*(a**t)

def sim_annealing(queens):
    state=queens.copy()
    steps=0
    for t in range(1000000000):
        T=schedule(t)
        attacks=h(queens)
        if T==0:
            return state,h(state),steps
        state=queens.copy()
        state[random.randint(0,7)]=random.randint(0,7)
        delE=h(state)-attacks
        #print(state)
        #print(h(state))
```

```python
def sim_annealing(queens):
        if delE<0:
            queens=state.copy()
            steps+=1
            #print(h(state))
            if h(state)==0:
                return queens, 0, steps
                    #print(neighbour)
        else:
            prob=e**(-delE/T)
            if random.random()<=prob:
                steps+=1
                queens=state.copy()

        #print(h(queens),attacks,queens,neighbour)


def main():
    success=0
    avg_steps=0
    for i in range(iterations):
        print(f"Board {i+1}:")
        queens=generate_board()
        ans,attacks,cost=first_choice(queens)
        print(queens,h(queens))
        print(ans,attacks,cost)
        avg_steps+=cost
        if attacks==0:
            success+=1
            print("Solved\n")
        else:
            print("Failed\n")
    avg_steps/=iterations
    print("===== First Choice =====")
    print(f"Success Rate={success/iterations*100}%    Average Steps Taken={avg_steps}\n")

    success=0
    avg_steps=0
    for i in range(iterations):
        print(f"Board {i+1}:")
        queens=generate_board()
        ans,attacks,cost=random_restart(queens)
        print(queens,h(queens))
        print(ans,attacks,cost)
        avg_steps+=cost
```

```python
        if attacks==0:
            success+=1
            print("Solved\n")
        else:
            print("Failed\n")
    avg_steps/=iterations
    print("===== Random Restart =====")
    print(f"Success Rate={success/iterations*100}%    Average Steps Taken={avg_steps}\n")

    success=0
    avg_steps=0
    for i in range(iterations):
        print(f"Board {i+1}:")
        queens=generate_board()
        ans,attacks,cost=sim_annealing(queens)
        print(queens,h(queens))
        print(ans,attacks,cost)
        avg_steps+=cost
        if attacks==0:
            success+=1
            print("Solved\n")
        else:
            print("Failed\n")
    avg_steps/=iterations
    print("===== Simulated Annealing =====")
    print(f"Success Rate={success/iterations*100}%    Average Steps Taken={avg_steps}\n")

if __name__=="__main__":
    main()
```

Output:

```
Board 1:
[6, 6, 6, 0, 7, 6, 2, 4] 11
[0, 5, 7, 1, 3, 6, 2, 4] 1 9
Failed

Board 2:
[3, 1, 2, 2, 1, 0, 3, 6] 9
[1, 5, 2, 5, 7, 0, 3, 6] 1 8
Failed

Board 3:
[2, 6, 4, 1, 7, 4, 6, 0] 4
[2, 5, 3, 1, 7, 4, 6, 0] 0 3
Solved

Board 4:
[2, 0, 0, 6, 7, 3, 3, 1] 7
[5, 2, 4, 7, 0, 6, 3, 1] 1 7
Failed

Board 5:
[1, 5, 5, 5, 3, 3, 0, 3] 9
[4, 2, 7, 5, 1, 6, 0, 3] 1 6
Failed

Board 6:
[6, 5, 3, 5, 4, 6, 5, 3] 9
[7, 4, 0, 0, 4, 6, 5, 3] 3 7
Failed

Board 7:
[7, 4, 0, 1, 4, 7, 6, 5] 9
[7, 0, 3, 1, 4, 2, 6, 3] 3 6
Failed

Board 8:
[7, 3, 7, 4, 7, 0, 5, 0] 8
[6, 3, 7, 7, 4, 0, 5, 0] 2 5
Failed

===== First Choice =====
Success Rate=12.5%    Average Steps Taken=6.375
```

```
Board 1:
[5, 4, 4, 0, 3, 4, 3, 2] 10
[4, 0, 7, 3, 1, 6, 2, 5] 0 14
Solved

Board 2:
[0, 1, 4, 4, 5, 4, 7, 5] 9
[3, 5, 7, 1, 6, 0, 2, 4] 0 12
Solved

Board 3:
[6, 1, 5, 6, 1, 7, 3, 1] 7
[5, 2, 4, 6, 0, 3, 1, 7] 0 6
Solved

Board 4:
[3, 3, 0, 3, 1, 3, 4, 5] 12
[3, 5, 7, 2, 0, 6, 4, 1] 0 25
Solved

Board 5:
[6, 0, 5, 4, 7, 1, 3, 2] 4
[3, 6, 4, 1, 5, 0, 2, 7] 0 35
Solved

Board 6:
[7, 1, 0, 7, 0, 5, 3, 3] 9
[1, 7, 5, 0, 2, 4, 6, 3] 0 8
Solved

Board 7:
[7, 5, 2, 7, 1, 5, 7, 0] 8
[1, 3, 5, 7, 2, 0, 6, 4] 0 7
Solved

Board 8:
[4, 4, 6, 2, 4, 5, 2, 5] 11
[2, 4, 1, 7, 5, 3, 6, 0] 0 9
Solved

===== Random Restart =====
Success Rate=100.0%    Average Steps Taken=14.5
```

```
Board 1:
[2, 3, 4, 2, 0, 0, 2, 5] 10
[2, 5, 1, 6, 0, 3, 7, 4] 0 135
Solved

Board 2:
[3, 6, 1, 1, 2, 6, 2, 0] 6
[2, 5, 1, 4, 7, 0, 6, 3] 0 419
Solved

Board 3:
[5, 6, 5, 0, 0, 5, 1, 1] 9
[4, 0, 7, 5, 2, 6, 1, 3] 0 102
Solved

Board 4:
[6, 2, 5, 5, 6, 3, 4, 1] 8
[6, 3, 1, 4, 7, 0, 2, 5] 0 203
Solved

Board 5:
[0, 1, 4, 1, 7, 4, 6, 4] 8
[2, 4, 6, 0, 3, 1, 7, 5] 0 68
Solved

Board 6:
[5, 1, 7, 6, 5, 5, 5, 0] 11
[5, 2, 0, 6, 4, 7, 1, 3] 0 115
Solved

Board 7:
[0, 3, 3, 6, 3, 4, 5, 2] 9
[4, 1, 3, 5, 7, 2, 0, 6] 0 104
Solved

Board 8:
[6, 1, 4, 0, 0, 2, 4, 3] 6
[2, 7, 3, 6, 0, 5, 1, 4] 0 189
Solved

===== Simulated Annealing =====
Success Rate=100.0%    Average Steps Taken=166.875
```